

AdaBoost Under the Hood: The Algorithmic Loop

A step-by-step mathematical deconstruction of Adaptive Boosting from scratch.

Index	X1	X2	Output (Y)
0	1	5	1
1	2	4	1
2	3	1	0
3	4	2	0
4	5	8	1
5	6	9	0
6	7	3	1
7	8	6	0
8	9	9	0
9	10	7	1

Step 1: Initialise Equal Data Weights

Index	X1	X2	Output (Y)	Initial Weight
1	0.25	0.38	A	0.1
2	0.67	0.81	B	0.1
3	0.12	0.95	A	0.1
4	0.44	0.56	B	0.1
5	0.89	0.22	A	0.1
6	0.33	0.77	B	0.1
7	0.58	0.14	A	0.1
8	0.91	0.49	B	0.1
9	0.29	0.63	A	0.1
10	0.71	0.35	B	0.1

$$w_i = \frac{1}{N}$$

Before training begins, every row is assigned a uniform weight. With $N = 10$ rows, each point is given a weight of 0.1. The model currently has no bias toward any specific data point.

Step 2: Train Base Learner 1 & Identify Errors

Index	X1	X2	Output (Y)	Initial Weight
0	0.25	0.38	A	0.1
1	0.67	0.81	B	0.1
2	0.12	0.95	A	0.1
3	0.44	0.56	B	0.1
4	0.89	0.22	A	0.1
5	0.33	0.77	B	0.1
6	0.58	0.14	A	0.1
7	0.91	0.49	B	0.1
8	0.29	0.63	A	0.1
10	0.71	0.35	B	0.1

The Model

We train a Decision Stump (a decision tree with a maximum depth of 1) on feature X1.

The Result

The weak learner is imperfect. It misclassifies three points (Rows 2, 6, and 8).

Step 3: Calculate Total Error Rate

Index	X1	X2	Output (Y)	Initial Weight
2	0.67	0.81	B	0.1
6	0.33	0.77	B	0.1
8	0.91	0.49	B	0.1

Total Error = Σ weights
of misclassified points

$$0.1 + 0.1 + 0.1 = 0.3$$

The error is not just '3 out of 10'. It is the sum of the weights of the incorrect predictions. Because the weights are currently uniform, the error is exactly 0.3.

Step 4: Compute Model Trustworthiness (α)

α (Alpha) dictates how much say this specific stump will have in the final prediction. A lower error yields a higher α .

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \text{Total Error}}{\text{Total Error}} \right)$$

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1 - 0.3}{0.3} \right) \approx 0.42$$

For Model 1, an error of 0.3 gives us an α of 0.42.

Step 5: Exponentially Penalise Mistakes

Initial Weight		Updated Weight
0.1	→	0.06
0.1	→	0.06
0.1	→	0.15
0.1	→	0.06
0.1	→	0.06
0.1	→	0.06
0.1	→	0.15
0.1	→	0.06
0.1	→	0.15
0.1	→	0.06
0.1	→	0.06

Why e^α ?

If α is high (a trustworthy model), we aggressively scale weights.

Incorrect Predictions:

$$W_{\text{new}} = W_{\text{old}} \times e^\alpha$$

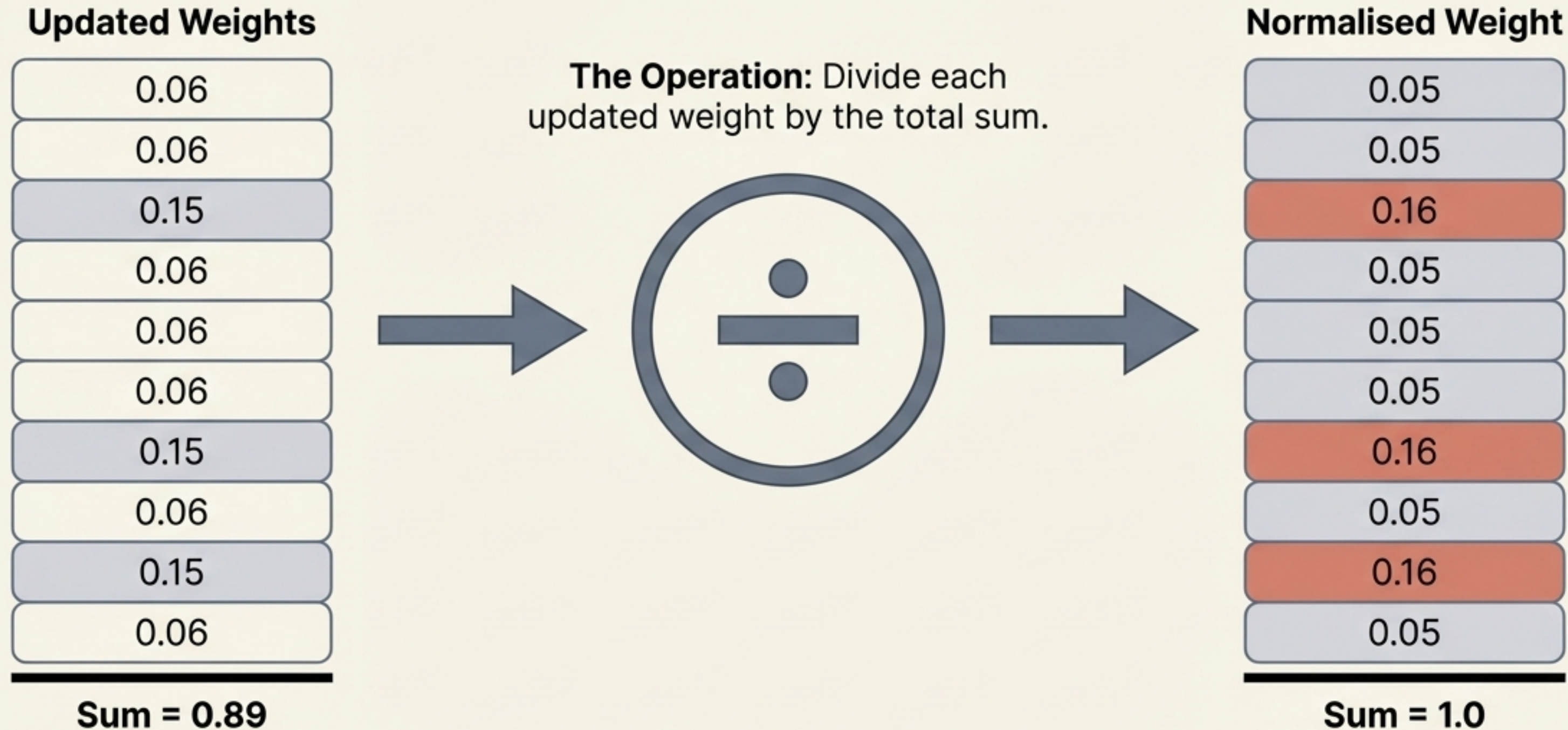
(Weight increases to force focus).

Correct Predictions:

$$W_{\text{new}} = W_{\text{old}} \times e^{-\alpha}$$

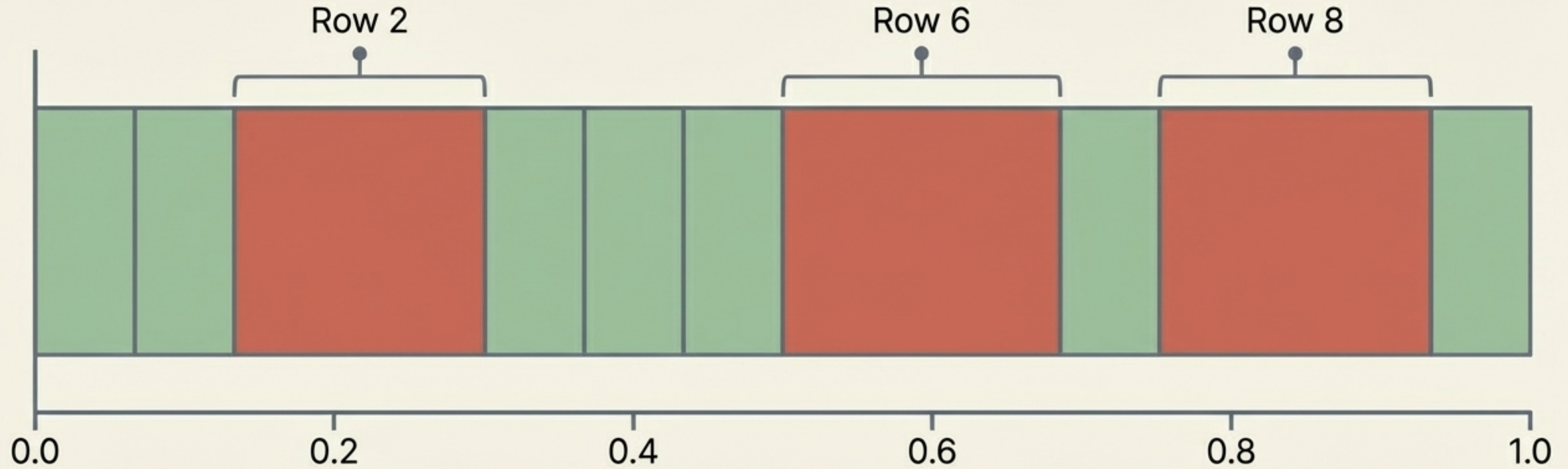
(Weight decreases as they are already solved).

Step 6: Normalise the Updated Weights



The Result: The misclassified rows (2, 6, 8) now hold 16% of the total weight each, while correct rows hold only 5% each.

Step 7: Create Bins for Resampling

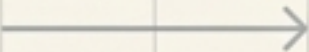


Explanation: We convert the normalised weights into cumulative upper and lower bounds. Because rows 2, 6, and 8 were misclassified, their 'bins' occupy nearly half of the total 0.0 – 1.0 space.

Step 8: Upsample via the 'Roulette Wheel'

The Action: Generate 10 random numbers between 0 and 1. Select the row whose bin the number falls into.

Random Number Generator
0.12
0.45
0.77
0.21
0.89
0.55
0.62
0.19
0.33
0.81

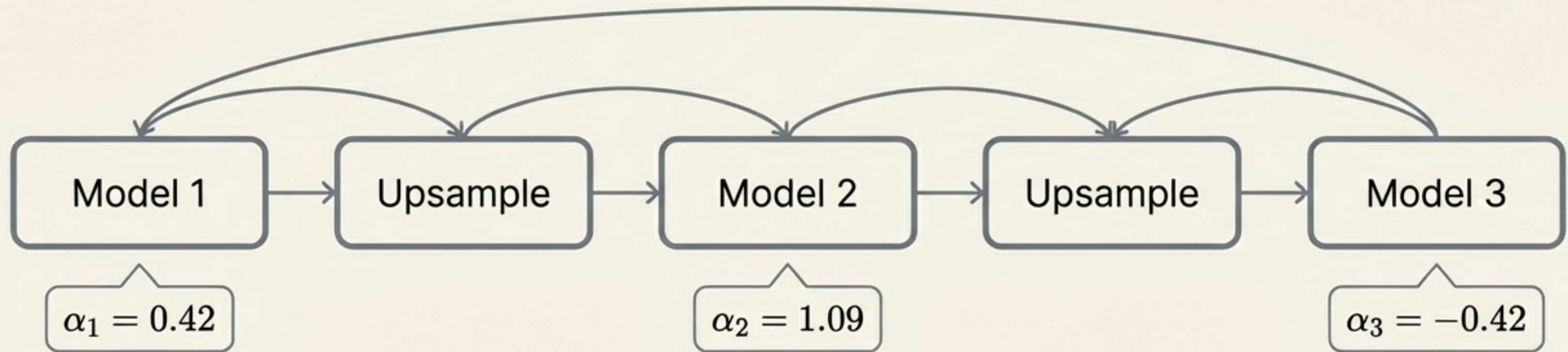


Upsampled Dataset

Original Index	X1	X2
0	...	...
2	...	...
2	...	...
4	...	...
6	...	...
6	...	...
6	...	...
8	...	...
8	...	...
9	...	...

The Result: The new dataset is exactly 10 rows long, but the previously misclassified points have been duplicated. The next model has no choice but to prioritise them.

Step 9: Reset, Train, and Repeat



- **Crucial Rule:**

Before training Base Learner 2 on the newly newly upsampled data, all row weights are reset to uniform ($w_i = 1/N$).

- **Iteration 2 Results:**

Makes only 1 error on the original data mapping.
Error = 0.1 $\rightarrow \alpha_2 = 1.09$.

- **Iteration 3 Results:**

Struggles on the shifted data, making 7 errors.
Error = 0.7 $\rightarrow \alpha_3 = -0.42$.

Practitioner Note: The Zero-Error Edge Case

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \text{Total Error}}{\text{Total Error}} \right)$$



The Danger & Math Failure

What if a Decision Stump makes zero mistakes on a small dataset?

Total Error = 0.

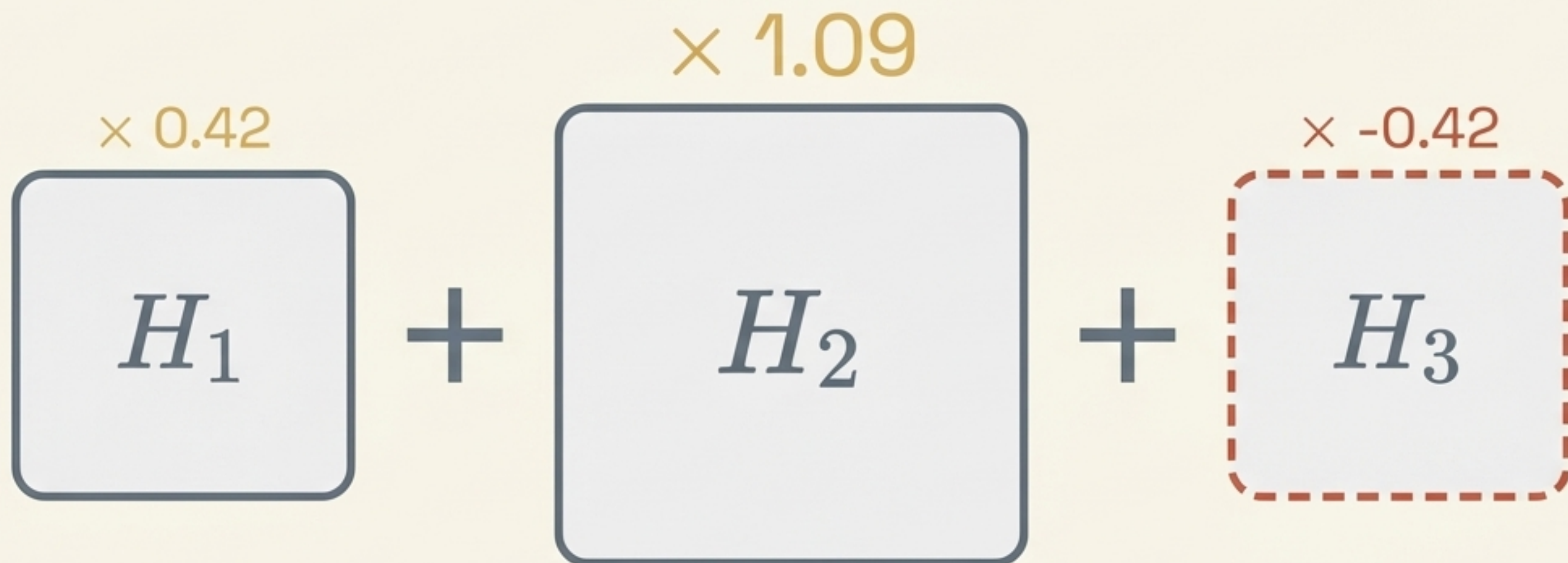
$\ln \left(\frac{1 - 0}{0} \right)$ results in a division by zero.

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \text{Total Error}}{\text{Total Error} + \epsilon} \right)$$

The Fix

Add a tiny epsilon value to the denominator in your code to prevent the calculation from breaking while still rewarding the perfect model with a massive α .

Step 10: The Weighted Majority Vote



Final Output Formula: $\text{Sign}\left(\sum \alpha_i \times H_i(x)\right)$

Data Requirement: To perform the math correctly, binary output predictions of '0' must be mapped to '-1'.

Explanation: Every model gets a vote, but not all votes are equal. The final prediction relies on the mathematical sign (positive or negative) of the weighted sum.

Predicting Unseen Data

Point (1, 5)

$$0.42(1) + 1.09(1) + (-0.42)(1)$$



$$= 1.09$$

Result: Positive → Output 1

Point (9, 9)

$$0.42(1) + 1.09(-1) + (-0.42)(-1)$$



$$= -0.25$$

Result: Negative → Output -1
(mapped back to 0)

Notice how heavily Model 2 ($\alpha = 1.09$) sways the final decision for the point (9, 9), successfully overriding the incorrect positive vote of Model 1. The ensemble is stronger than the individual.